



MLOps Mission: Industrialization & Deployment

Report

Project: Scouts Tunisia DSS

Phase: Week S12 - MLOps Approach Integration

This report outlines exactly how each validation criterion requested in the mission brief was successfully implemented, resolved, and deployed.

✓ 1. Experiment Tracking (MLflow)

Objective: Track parameters, metrics, and ensure at least two runs are comparable.

- **Implementation:** Developed the script `mlops_train.py` to seamlessly execute the At-Risk Unit detection model.
- **Validation:** MLflow is now successfully instantiated locally via its default FileStore. The experiment **Scouts_AtRisk_Classification** logged two distinct runs (e.g., `lyrical-wolf` and `unequaled-mink`) ensuring full comparability in the MLflow UI's Table tab.
- **Details Tracked:** Input data schemas, Hyper-parameters (`n_estimators` , `max_depth`), and Performance metrics (`accuracy=100%`).

✓ 2. Automated Training Pipeline

Objective: Ensure an end-to-end pipeline (preprocessing → training → evaluation → saving) that requires no manual intervention.

- **Implementation:** The `mlops_train.py` handles the complete cycle. By running a single command (`python mlops_train.py`), the script automatically scales features via deterministic logic, trains the Random Forest classifier, scores the test set, and pipes the output cleanly to the MLflow logger.

✓ 3. Model Management

Objective: Save, version, and ensure previous models are accessible.

- **Implementation:** Inside `mlops_train.py` , the model is officially registered to the MLflow Model Registry as `AtRiskClassifier` (Version 1).
- **Validation:** MLflow strictly archives each version. As new training scripts are triggered, new iterations are saved to the persistent `mlruns/` registry to ensure rollback capability.

✓ 4. Model Serving API (FastAPI)

Objective: Build a functional prediction API exposing the `/predict` endpoint.

- **Implementation:** Authored `mlops_api.py` utilizing the modern **FastAPI** framework.
- **Validation:** The service launches on port 8000 and successfully takes dynamically-typed JSON structures (Members, Leaders, Participation) and outputs the classification directly from the `models/at_risk_model.pkl` artifact.

✓ 5. Containerization (Included Docker & Docker Compose)

Objective: Ensure the application safely runs inside Docker.

- **Implementation:** The architecture was heavily isolated into two specific Dockerfiles:
 - `Dockerfile.api` : Encapsulating the FastAPI prediction server.

- `Dockerfile.webapp` : Encapsulating the main Flask dashboard ecosystem.
- **Validation:** Built a massive `docker-compose.yml` orchestrator. We solved a critical system-file-lock exception in the Windows background by configuring a highly specific `.dockerignore` file. The environment now boots cleanly mapped to `localhost:5000` via `docker-compose up -d --build`.

✓ 6. Code Quality

Objective: Clean, modular, well-commented execution.

- **Implementation:** We definitively decoupled the monolithic Flask app into segmented zones. Training logic is safely relegated to `mlops_train.py`, API exposure belongs strictly to `mlops_api.py`, and the legacy UI is preserved in the main app.

✓ 7. Web App Integration (Frontend to API)

Objective: Establish the End-to-End flow: UI → API → Model → Result.

- **Implementation:** Expanded `ml_app/app.py` by adding an internal `/mlops/predict` route that elegantly proxies requests to the external FastAPI instance mapping.
- **Validation:** We modified the HTML frontend (`index.html`) by integrating a brand new **Objective 13** (🚀 **MLOps API Test**) module. You can visually input variables in the browser, triggering the complete REST cycle to return the risk classification over the network flawlessly.